

# Operating System Notes

## Unit – 1: Introduction

### INDEX:

| <u>S.NO.</u> | <u>TOPIC</u>                             | <u>P.NO.</u> |
|--------------|------------------------------------------|--------------|
| 1            | Operating System Objective and Functions | 1            |
| 2            | Evolution of Operating System            | 3            |
| 3            | Components of Operating System           | 9            |
| 4            | Services of Operating System             | 16           |
| 5            | Operating System Structure               | 19           |
| 6            | Computer System Structure                | 32           |
| 7            | Interrupts                               | 23           |
| 8            | System Call                              | 23           |
| 9            | Modes of Operation                       | 29           |
| 10           | System Programs in OS                    | 30           |

## **What is an Operating System?**

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

## **Operating System Definition**

- No universally accepted definition
- Operating System exists to offer a reasonable way to solve the problem of creating a usable computing system.
- The fundamental goal of computer system is to execute system programs and to make solving user programs easier.
- Since bare hardware alone is not particularly easy to use, application programs are developed.
- A simple view point is that it includes everything a vendor ships when you order the Operating System.
- The features that are included vary greatly across systems.
- A more common definition: “The one program running at all times on the computer” is the **kernel**.
- Along with the Kernel, there are two different types of programs : System Programs, Application Programs
- The emergence of mobile devices have resulted in an increase in the number of features that constitute an operating system.
- The mobile operating systems often include not only a core kernel, but also a middleware.

## **Objectives of Operating System**

The objectives of the operating system are –

- To make the computer system *convenient to use* in an efficient manner.
- To *hide the details of the hardware* resources from the users.
- To *provide users a convenient interface* to use the computer system.
- To act as an *intermediary* between the hardware and its users, *making it easier for the users to access and use other resources*.

- To *manage the resources* of a computer system.
- To *keep track* of who is using which resource, granting resource requests, and mediating conflicting requests from different programs and users.
- To provide *efficient and fair sharing of resources* among users and programs.

## Characteristics/Functions of Operating System

Here is a list of some of the most prominent characteristic features of Operating Systems –

- **Memory Management** – Keeps track of the primary memory, i.e. what part of it is in use by whom, what part is not in use, etc. and allocates the memory when a process or program requests it.
- **Processor Management** – Allocates the processor (CPU) to a process and deallocates the processor when it is no longer required.
- **Device Management** – Keeps track of all the devices. This is also called I/O controller that decides which process gets the device, when, and for how much time.
- **File Management** – Allocates and de-allocates the resources and decides who gets the resources.
- **Security** – Prevents unauthorized access to programs and data by means of passwords and other similar techniques.
- **Job Accounting** – Keeps track of time and resources used by various jobs and/or users.
- **Control over System Performance** – Records delays between the request for a service and from the system.
- **Interaction with the Operators** – Interaction may take place via the console of the computer in the form of instructions. The Operating System acknowledges the same, does the corresponding action, and informs the operation by a display screen.
- **Error-detecting Aids** – Production of dumps, traces, error messages, and other debugging and error-detecting methods.
- **Coordination between Other Software and Users** – Coordination and assignment of compilers, interpreters, assemblers, and other software to the various users of the computer systems.

## Evolution of the Operating System / Types of Operating System

- The computer network has many resources such as software and hardware that are mandatory to finish the task. Generally, the required resources are file storage, CPU, memory, input and output devices and so on. The operating system acts as the controller of all the above-mentioned resources and assigns them with specific programs executed to perform the task. Hence, the operating system is a resource manager that handles the resource as a user view and system view. The evolution of the operating system is marked from the programming of punch cards to training machines to speak and interpret any language.

# Various Evolution of the Operating System

- The various evolution of an operating system are given below

## Serial Processing:

- Early computer from late 1940 to the mid 1950's.
- The programmer interacted directly with the computer hardware.
- These machine are called bare machine as they don't have OS.
- Every computer system is programmed in its machine language.
- Uses Punch Card, paper tapes and language translator

**These system presented two major problems.**

1. Scheduling
2. Set up time

**Scheduling:** Used signup sheet to reserve machine time. A user may sign up for an hour but finishes his job in 45 minutes. This would result in wasted computer idle time, also the user might run into the problem not finish his job in allotted time.

**Set up time:** A single program involves:

- Loading compiler and source program in memory
  - saving the compiled program (object code)
  - Loading and linking together object program and common function
- Each of these steps involves the mounting or dismounting tapes on setting up punch cards. If an error occur user had to go the beginning of the set up sequence.
  - Thus, a considerable amount of time is spent in setting up the program to run.
  - This mode of operation is turned as serial processing, reflecting the fact that users access the computer in series.

## The Batch System

- Early computers were very expensive, and therefore it was important to maximize processor utilization.
  - The wasted time due to scheduling and setup time in Serial Processing was unacceptable.
  - To improve utilization, the concept of a batch operating system was developed.
  - Batch is defined as a group of jobs with similar needs. The operating system allows users to form batches.
- Computer executes each batch sequentially, processing all jobs of a batch considering them as a single process called batch processing.

**The central idea behind the simple batch-processing scheme is:**

- Jobs were scheduled and submitted on cards and tapes. Then sequentially executed on the monitors by using Job Control Language. The first computers are used in the process of the batch operating process made the computer batch of jobs without any pause or stop. The program is written in the punch cards and then copied to the processing unit of the tape. When the computer completed a single job, it instantly begins the next task on the tape. Professional operators are trained to communicate with the machine where the users dropped the jobs and fetched back to pick the results after the job is executed.

- Though it is uncomfortable for the users it is made to keep the expensive computer as busy up to the extent by running a leveraged stream of jobs. The protection of memory doesn't allow the memory area comprises the monitor to altered and the timer protects the job from monopolizing the system. The processor sustains as idle when the input and output devices are in use by the bad utilization of CPU time.

### Multi-programmed Batch System:

- A single program cannot keep either CPU or I/O devices busy at all times.
- Multiprogramming increases CPU utilization by organizing jobs in such a manner that CPU has always one job to execute.
- If computer is required to run several programs at the same time, the processor could be kept busy for the most of the time by switching its attention from one program to the next.
- Additionally I/O transfer could overlap the processor activity i.e, while one program is awaiting for an I/O transfer, another program can use the processor.
- So CPU never sits idle or if comes in idle state then after a very small time it is again busy. This is illustrated in fig below.

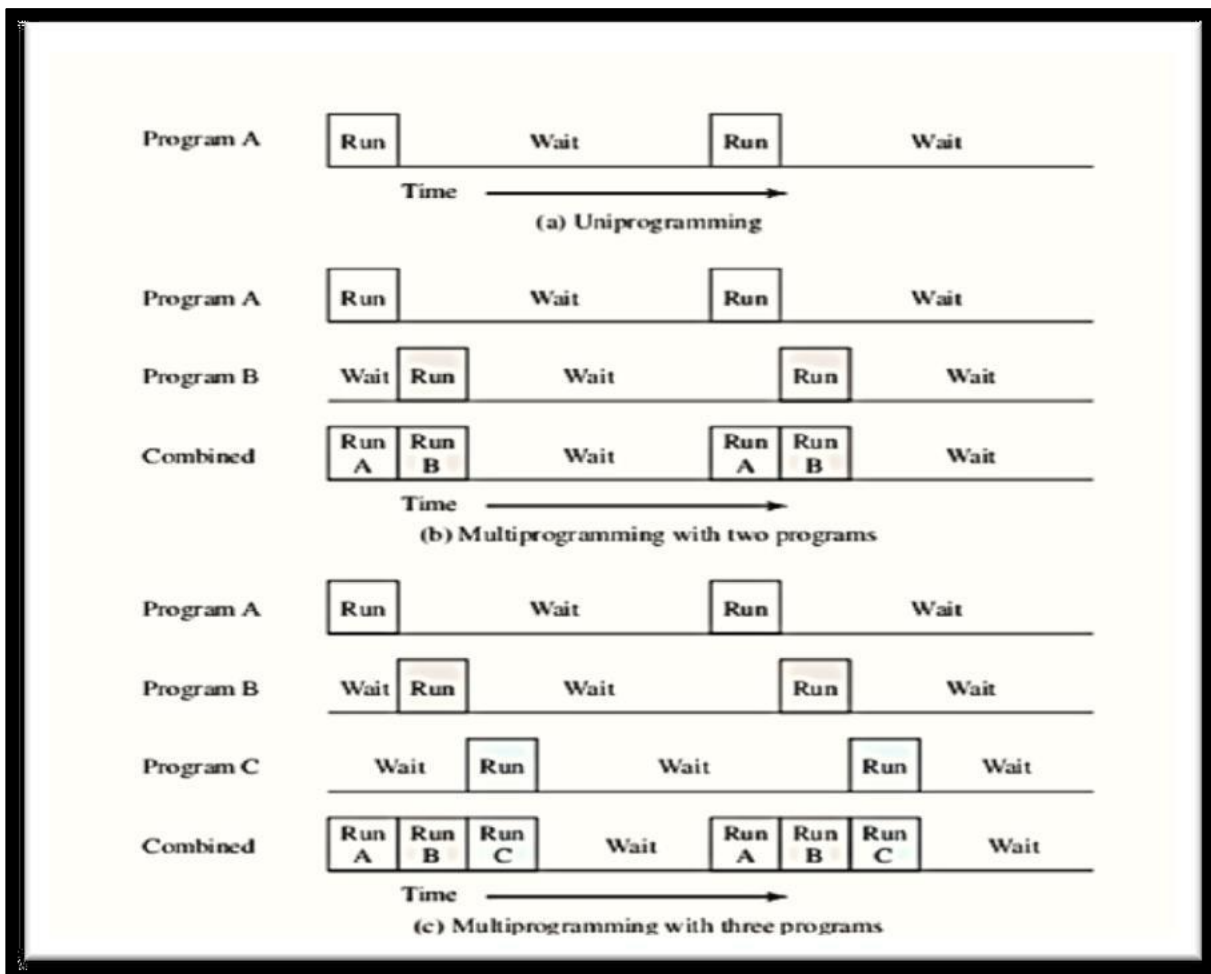


Fig. Multiprogramming example

### Multitasking or Time Sharing System:

- Multiprogramming didn't provide the user interaction with the computer system.
- Time sharing or Multitasking is a logical extension of Multiprogramming that provides user interaction.

- There are more than one user interacting the system at the same time
- The switching of CPU between two users is so fast that it gives the impression to user that he is only working on the system but actually it is shared among different users.
- CPU bound is divided into different time slots depending upon the number of users using the system.
- Just as multiprogramming allows the processor to handle multiple batch jobs at a time, multiprogramming can also be used to handle multiple interactive jobs. In this latter case, the technique is referred to as time sharing, because processor time is shared among multiple users
- A multitasking system uses CPU scheduling and multiprogramming to provide each user with a small portion of a time shared computer. Each user has at least one separate program in memory.
- Multitasking are more complex than multiprogramming and must provide a mechanism for jobs synchronization and communication and it may ensure that system does not go in deadlock.
- Although batch processing is still in use but most of the system today available uses the concept of multitasking and Multiprogramming.

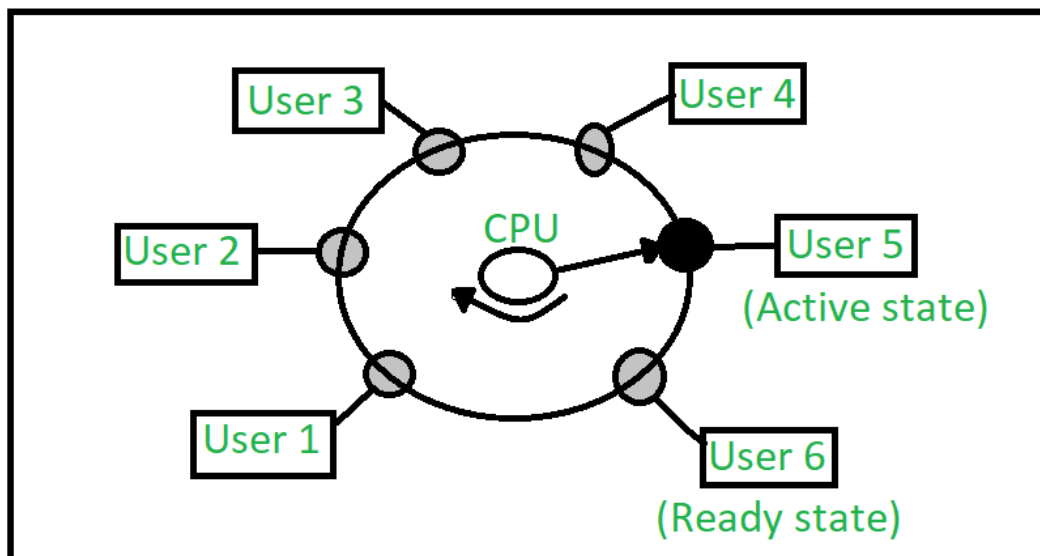


Fig. Time Sharing example

**Distributed Operating System** is one of the important type of operating system.

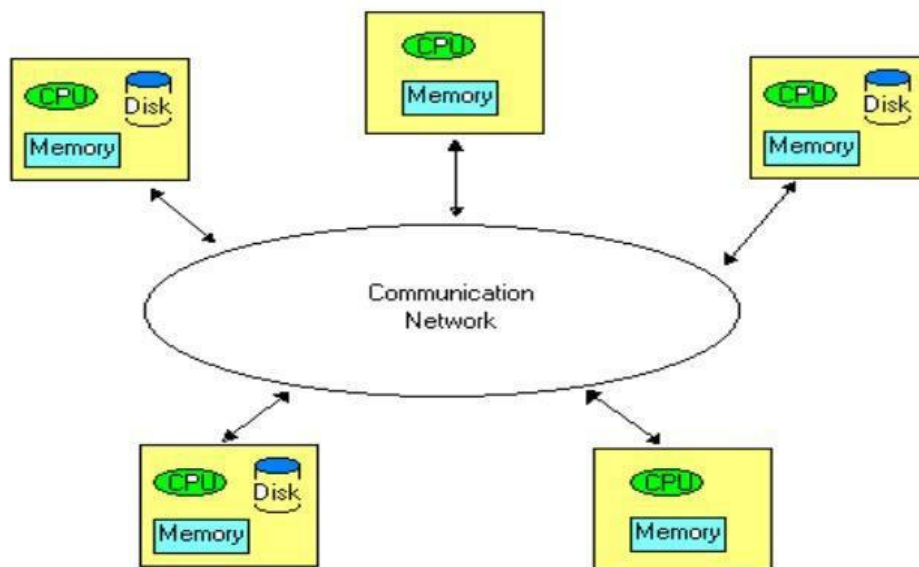
- Multiple central processors are used by Distributed systems to serve multiple real-time applications and multiple users. Accordingly, Data processing jobs are distributed among the processors.
- Processors communicate with each other through various communication lines (like high-speed buses or telephone lines). These are known as **loosely coupled systems** or distributed systems. Processors in this system may vary in size and function. They are referred as sites, nodes, computers, and so on.

#### Advantages

- With resource sharing facility, a user at one site may be able to use the resources available at another.
- Speedup the exchange of data with one another via electronic mail.
- Failure of one site in a distributed system doesn't affect the others, the remaining sites can potentially continue operating.
- Better service to the customers.

- Reduction of the load on the host computer.
- Reduction of delays in data processing.

## Architecture of Distributed OS



### Features of Distributed Operating System

#### 1. Connecting Users and Resources:

- The main goal of a distributed system is to make it easy for users to access remote resources, and to share them with other users in a controlled manner. Resources can be virtually anything; typical examples of resources are printers, storage facilities, data, files, web pages, and networks. There are many reasons for sharing resources. One reason is economics.

#### 2. Transparency:

An important goal of a distributed system is to hide the fact that its process and resources are physically distributed across multiple computers. A distributed system that is capable of presenting itself to users and applications such that it is only a single computer system is called transparent. The concept of transparency can be applied to many aspects of a distributed system as shown in table.

### Different Forms of Transparency

| S.NO. | TRANSPARENCY | DESCRIPTION                          |
|-------|--------------|--------------------------------------|
| (1)   | Access       | Hide data representation.            |
| (2)   | Location     | Hide location                        |
| (3)   | Migration    | Move place information.              |
| (4)   | Relocation   | Hide moved place relocation.         |
| (5)   | Replication  | Hide that a resource is replication. |
| (6)   | Concurrency  | Shared data bases access             |
| (7)   | Failure      | Hide fact about resource failure.    |
| (8)   | Persistence  | Hide fact about memory location.     |

3. **Openness:** Another important goal of distributed systems is openness. An open distributed system is a system that offers services in standards that describable the syntax and semantics of those service instances, standard rules in computer networks control the format, content, and meaning of messages sent and received. Such rules are formalized in the protocols. In distributed systems, services are typically specified through interfaces, often called interface definition languages (IDL). Interface definitions written in IDL almost always capture only the syntax of services. They accurately specify the names of functions that are available with the types of parameters, return values, possible exceptions that can be raised and so on.

4. **Scalability:** The uncertain trend in distributed systems is towards larger systems. This observation has implications for distributed file system design. Algorithms that work well for systems with 100 machines can work for systems with 1000 machines and none at all for systems with 10, 000 machines. for starters, the centralized algorithm does not scale well. If opening a file requires contacting a single centralized server to record the fact that the file is open then the server will eventually become a bottleneck as the system grows.

5. **Reliability:** The main goal of building distributed systems was to make them more reliable than single processor systems. The idea is that if some machine goes down, some other machine gets used to it. In other words, theoretically the reliability of the overall system can be a Boolean OR of the component reliability. For example, with four file servers, each with a 0.95 chance of being up at any instant, the probability of all four being down simultaneously is 0.000006, so the probability of at least one being available is  $(1-0.000006) = 0.999994$ , far better than any individual server.

6. **Performance:** Building a transparent, flexible, reliable distributed system is useless if it is slow like molasses. In particular application on a distributed system, it should not deteriorate better than running some application on a single processor. Various performance metrics can be used. Response time is one, but so are throughput, system utilization, and amount of network capacity consumed. Furthermore, The results of any benchmark are often highly dependent on the nature of the benchmark. A benchmark involves a large number of independent highly CPU-bound computations which give radically different results than a benchmark that consists of scanning a single large file for same pattern.

## Real Time operating System

A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment. The time taken by the system to respond



to an input and display of required updated information is termed as the **response time**. So in this method, the response time is very less as compared to online processing.

Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application. A real-time operating system must have well-defined, fixed time constraints, otherwise the system will fail. For example, Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

There are two types of real-time operating systems.

#### **Hard real-time systems**

Hard real-time systems guarantee that critical tasks complete on time. In hard real-time systems, secondary storage is limited or missing and the data is stored in ROM. In these systems, virtual memory is almost never found.

#### **Soft real-time systems**

Soft real-time systems are less restrictive. A critical real-time task gets priority over other tasks and retains the priority until it completes. Soft real-time systems have limited utility than hard real-time systems. For example, multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers, etc.

## **Components of Operating System**

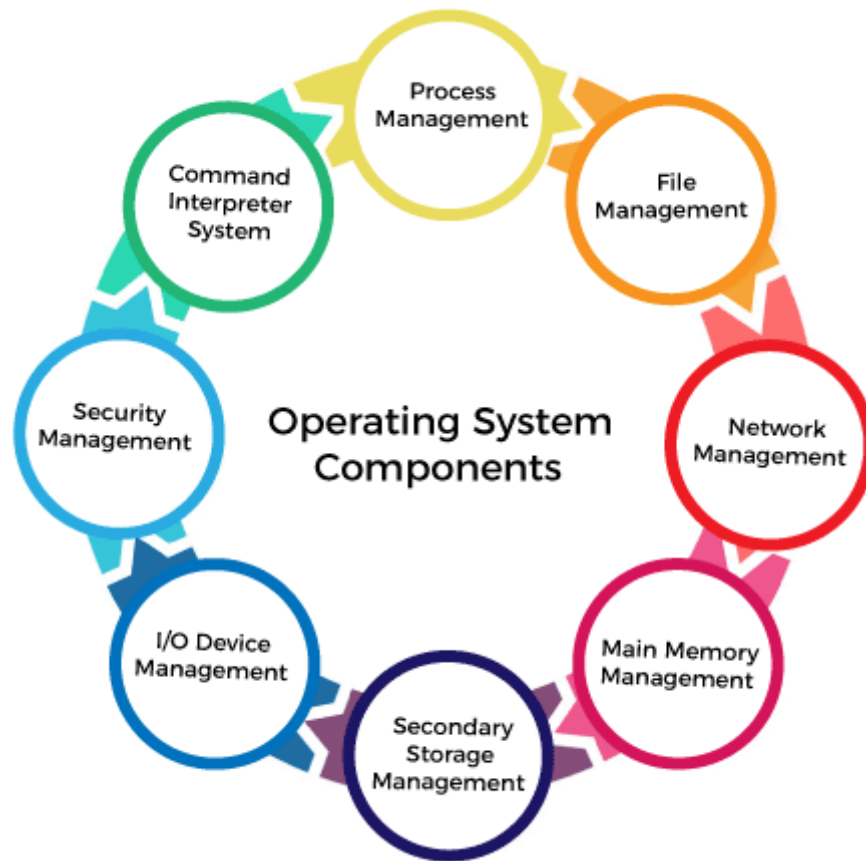
An operating system is a large and complex system that can only be created by partitioning into small parts. These pieces should be a well-defined part of the system, carefully defining inputs, outputs, and functions.

Although Windows, Mac, UNIX, Linux, and other OS do not have the same structure, most operating systems share similar OS system components, such as file, memory, process, I/O device management.

The components of an operating system play a key role to make a variety of computer system parts work together. There are the following components of an operating system, such as:

1. Process Management
2. File Management
3. Network Management
4. Main Memory Management
5. Secondary Storage Management
6. I/O Device Management
7. Security Management
8. Command Interpreter System

Operating system components help you get the correct computing by detecting CPU and memory hardware errors.



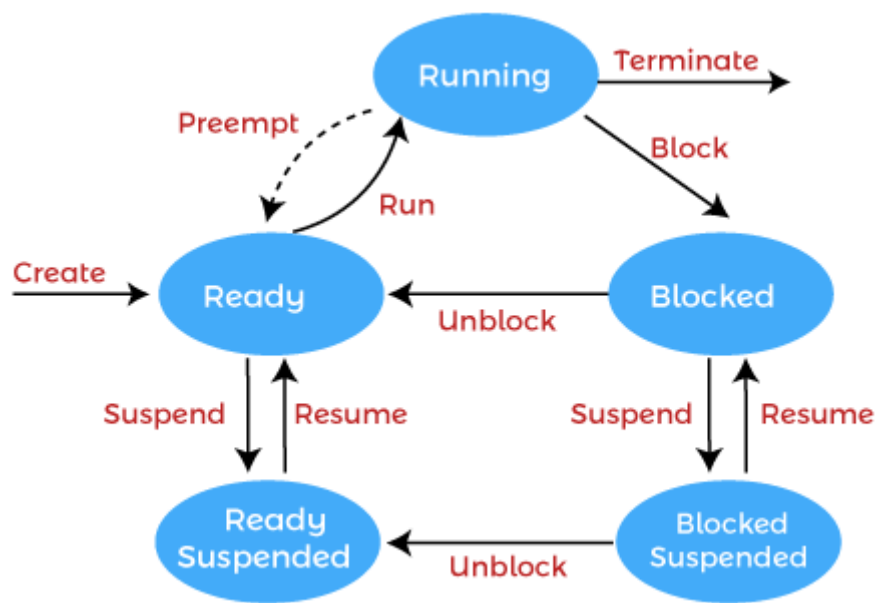
## Process Management

The process management component is a procedure for managing many processes running simultaneously on the operating system. Every running software application program has one or more processes associated with them.

For example, when you use a search engine like Chrome, there is a process running for that browser program.

Process management keeps processes running efficiently. It also uses memory allocated to them and shutting them down when needed.

The execution of a process must be sequential so, at least one instruction should be executed on behalf of the process.



### Functions of process management

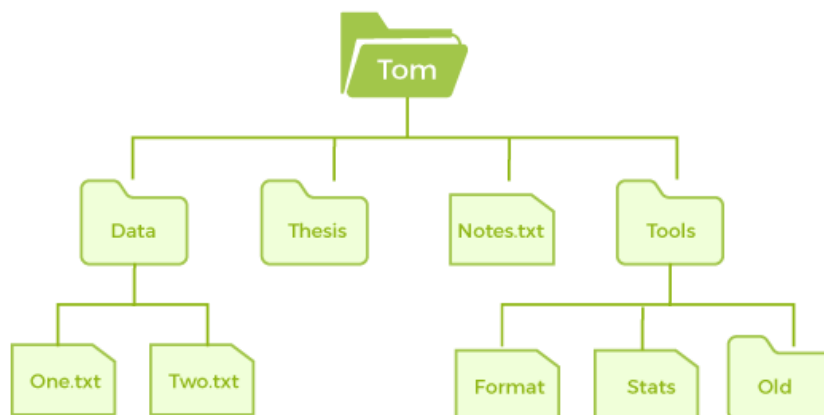
Here are the following functions of process management in the operating system, such as:

- Process creation and deletion.
- Suspension and resumption.
- Synchronization process
- Communication process

NOTE: OS facilitates an exchange of information between processes executing on the same or different systems.

### File Management

A file is a set of related information defined by its creator. It commonly represents programs (both source and object forms) and data. Data files can be alphabetic, numeric, or alphanumeric.



## Function of file management

The operating system has the following important activities in connection with file management:

- File and directory creation and deletion.
- For manipulating files and directories.
- Mapping files onto secondary storage.
- Backup files on stable storage media.

## Network Management

Network management is the process of administering and managing computer networks. It includes performance management, provisioning of networks, fault analysis, and maintaining the quality of service.

### Computer Networks

When we hook up computers together using data communication facilities, we call this a computer network.



A distributed system is a collection of computers or processors that never share their memory and clock. In this type of system, all the processors have their local memory, and the processors communicate with each other using different communication cables, such as fibre optics or telephone lines.

The computers in the network are connected through a communication network, which can configure in many different ways. The network can fully or partially connect in network management, which helps users design routing and connection strategies that overcome connection and security issues.

## Functions of Network management

Network management provides the following functions, such as:

- Distributed systems help you to various computing resources in size and function. They may involve minicomputers, microprocessors, and many general-purpose computer systems.

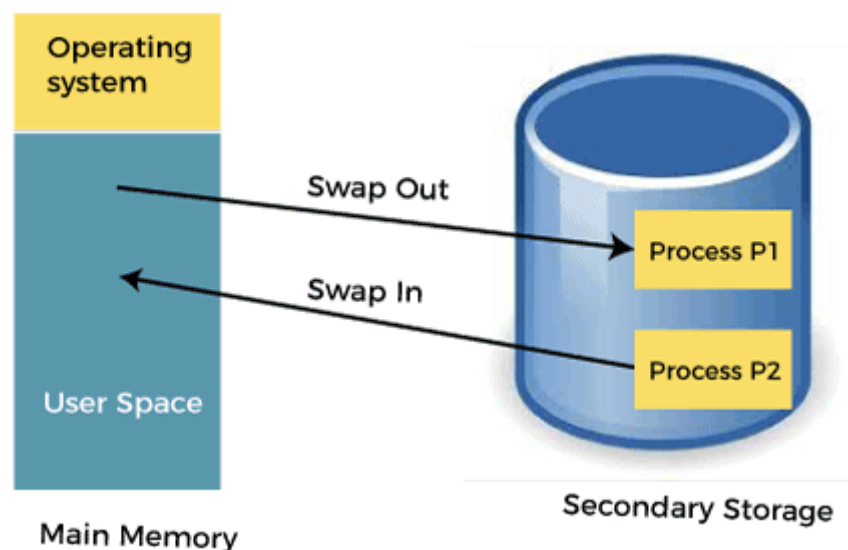
- A distributed system also offers the user access to the various resources the network shares.
- It helps to access shared resources that help computation to speed up or offers data availability and reliability.

## Main Memory management

Main memory is a large array of storage or bytes, which has an address. The memory management process is conducted by using a sequence of reads or writes of specific memory addresses.

It should be mapped to absolute addresses and loaded inside the memory to execute a program. The selection of a memory management method depends on several factors.

However, it is mainly based on the hardware design of the system. Each algorithm requires corresponding hardware support. Main memory offers fast storage that can be accessed directly by the CPU. It is costly and hence has a lower storage capacity. However, for a program to be executed, it must be in the main memory.



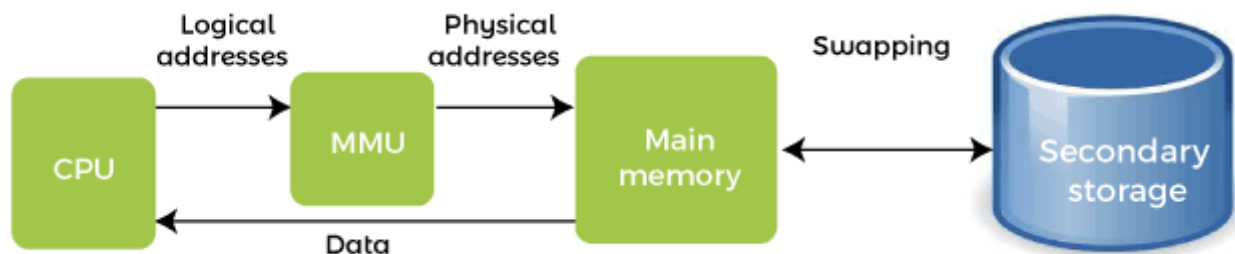
## Functions of Memory management

An Operating System performs the following functions for Memory Management in the operating system:

- It helps you to keep track of primary memory.
- Determine what part of it are in use by whom, what part is not in use.
- In a multiprogramming system, the OS decides which process will get memory and how much.
- Allocates the memory when a process requests.
- It also de-allocates the memory when a process no longer requires or has been terminated.

## Secondary-Storage Management

The most important task of a computer system is to execute programs. These programs help you to access the data from the main memory during execution. This memory of the computer is very small to store all data and programs permanently. The computer system offers secondary storage to back up the main memory.



Today modern computers use hard drives/SSD as the primary storage of both programs and data. However, the secondary storage management also works with storage devices, such as USB flash drives and CD/DVD drives. Programs like assemblers and compilers are stored on the disk until it is loaded into memory, and then use the disk is used as a source and destination for processing.

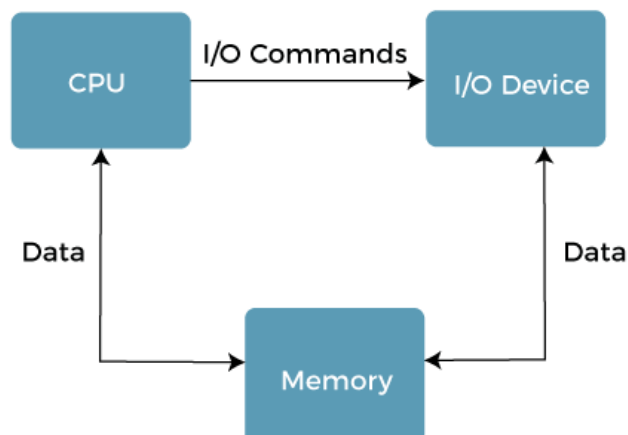
### Functions of Secondary storage management

Here are some major functions of secondary storage management in the operating system:

- Storage allocation
- Free space management
- Disk scheduling

## I/O Device Management

One of the important use of an operating system that helps to hide the variations of specific hardware devices from the user.



## Functions of I/O management

The I/O management system offers the following functions, such as:

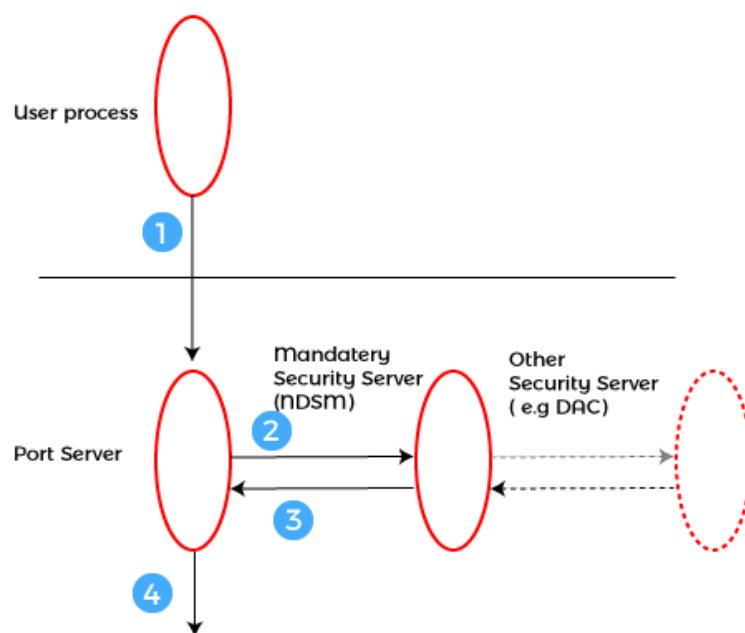
- It offers a buffer caching system
- It provides general device driver code
- It provides drivers for particular hardware devices.
- I/O helps you to know the individualities of a specific device.

**NOTE:** The user's program can't execute I/O operations directly. The operating system should provide some medium to perform this.

## Security Management

The various processes in an operating system need to be secured from other activities. Therefore, various mechanisms can ensure those processes that want to operate files, memory CPU, and other hardware resources should have proper authorization from the operating system.

Security refers to a mechanism for controlling the access of programs, processes, or users to the resources defined by computer controls to be imposed, together with some means of enforcement.

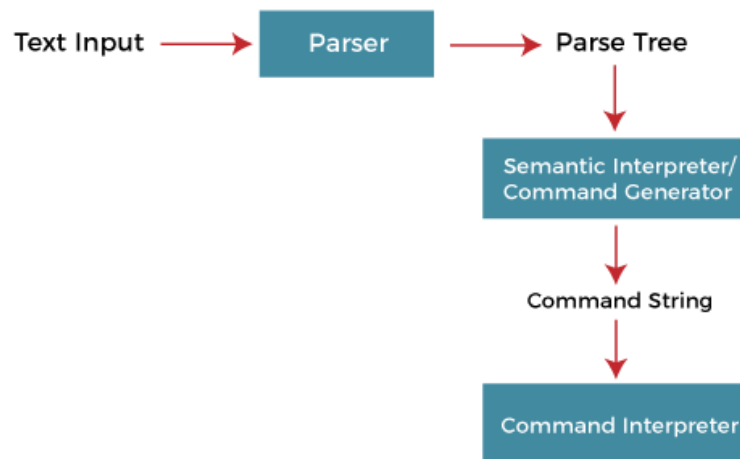


For example, memory addressing hardware helps to confirm that a process can be executed within its own address space. The time ensures that no process has control of the CPU without renouncing it. Lastly, no process is allowed to do its own I/O to protect, which helps you to keep the integrity of the various peripheral devices.

Security can improve reliability by detecting latent errors at the interfaces between component subsystems. Early detection of interface errors can prevent the foulness of a healthy subsystem by a malfunctioning subsystem. An unprotected resource cannot misuse by an unauthorized or incompetent user.

## Command Interpreter System

One of the most important components of an operating system is its command interpreter. The command interpreter is the primary interface between the user and the rest of the system.



Many commands are given to the operating system by control statements. A program that reads and interprets control statements is automatically executed when a new job is started in a batch system or a user logs in to a time-shared system. This program is variously called.

- The control card interpreter,
- The command-line interpreter,
- The shell (in UNIX), and so on.

Its function is quite simple, get the next command statement, and execute it. The command statements deal with process management, I/O handling, secondary storage management, main memory management, file system access, protection, and networking.

## Services of Operating System

An Operating System provides services to both the users and to the programs.

- It provides programs an environment to execute.
- It provides users the services to execute the programs in a convenient manner.

Following are a few common services provided by an operating system –

- Program execution



- I/O operations
- File System manipulation
- Communication
- Error Detection
- Resource Allocation
- Protection

## Program execution

Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc. Each of these activities is encapsulated as a process.

A process includes the complete execution context (code to execute, data to manipulate, registers, OS resources in use). Following are the major activities of an operating system with respect to program management –

- Loads a program into memory.
- Executes the program.
- Handles program's execution.
- Provides a mechanism for process synchronization.
- Provides a mechanism for process communication.
- Provides a mechanism for deadlock handling.

## I/O Operation

An I/O subsystem comprises of I/O devices and their corresponding driver software. Drivers hide the peculiarities of specific hardware devices from the users.

An Operating System manages the communication between user and device drivers.

- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required.

## File system manipulation

A file represents a collection of related information. Computers can store files on the disk (secondary storage), for long-term storage purpose. Examples of storage media include magnetic tape, magnetic disk and optical disk drives like CD, DVD. Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.

A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions. Following are the major activities of an operating system with respect to file management –

- Program needs to read a file or write a file.
- The operating system gives the permission to the program for operation on file.
- Permission varies from read-only, read-write, denied and so on.
- Operating System provides an interface to the user to create/delete files.
- Operating System provides an interface to the user to create/delete directories.

- Operating System provides an interface to create the backup of file system.

## **Communication**

In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, the operating system manages communications between all the processes. Multiple processes communicate with one another through communication lines in the network.

The OS handles routing and connection strategies, and the problems of contention and security. Following are the major activities of an operating system with respect to communication –

- Two processes often require data to be transferred between them
- Both the processes can be on one computer or on different computers, but are connected through a computer network.
- Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

## **Error handling**

Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware. Following are the major activities of an operating system with respect to error handling –

- The OS constantly checks for possible errors.
- The OS takes an appropriate action to ensure correct and consistent computing.

## **Resource Management**

In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job. Following are the major activities of an operating system with respect to resource management –

- The OS manages all kinds of resources using schedulers.
- CPU scheduling algorithms are used for better utilization of CPU.

## **Protection**

Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.

Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system. Following are the major activities of an operating system with respect to protection –

- The OS ensures that all access to system resources is controlled.
- The OS ensures that external I/O devices are protected from invalid access attempts.
- The OS provides authentication features for each user by means of passwords.

# Operating-System Design and Implementation

## Design Goals

**Requirements** define properties which the finished system must have, and are a necessary first step in designing any large complex system.

**User requirements** are features that users care about and understand, and are written in commonly understood vernacular. They generally do not include any implementation details, and are written similar to the product description one might find on a sales brochure or the outside of a shrink-wrapped box.

**System requirements** are written for the developers, and include more details about implementation specifics, performance requirements, compatibility constraints, standards compliance, etc. These requirements serve as a "contract" between the customer and the developers, ( and between developers and subcontractors ), and can get quite detailed.

Requirements for operating systems can vary greatly depending on the planned scope and usage of the system. ( Single user / multi-user, specialized system / general purpose, high/low security, performance needs, operating environment, etc. )

## Mechanisms and Policies

- Policies determine *what* is to be done. Mechanisms determine *how* it is to be implemented.
- If properly separated and implemented, policy changes can be easily adjusted without re-writing the code, just by adjusting parameters or possibly loading new data / configuration files. For example the relative priority of background versus foreground tasks.

## Implementation

- Traditionally OSES were written in assembly language. This provided direct control over hardware-related issues, but inextricably tied a particular OS to a particular HW platform.
- Recent advances in compiler efficiencies mean that most modern OSES are written in C, or more recently, C++. Critical sections of code are still written in assembly language, ( or written in C, compiled to assembly, and then fine-tuned and optimized by hand from there. )
- Operating systems may be developed using **emulators** of the target hardware, particularly if the real hardware is unavailable ( e.g. not built yet ), or not a suitable platform for development, ( e.g. smart phones, game consoles, or other similar devices. )

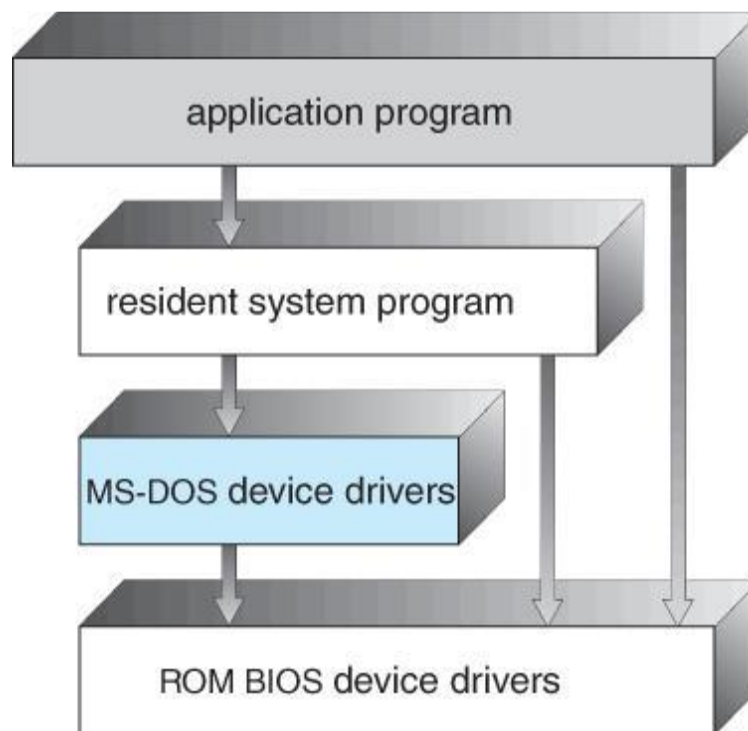
# Operating-System Structure

For efficient performance and implementation an OS should be partitioned into separate subsystems, each with carefully defined tasks, inputs, outputs, and performance characteristics. These subsystems can then be arranged in various architectural configurations:

## **Simple Structure**

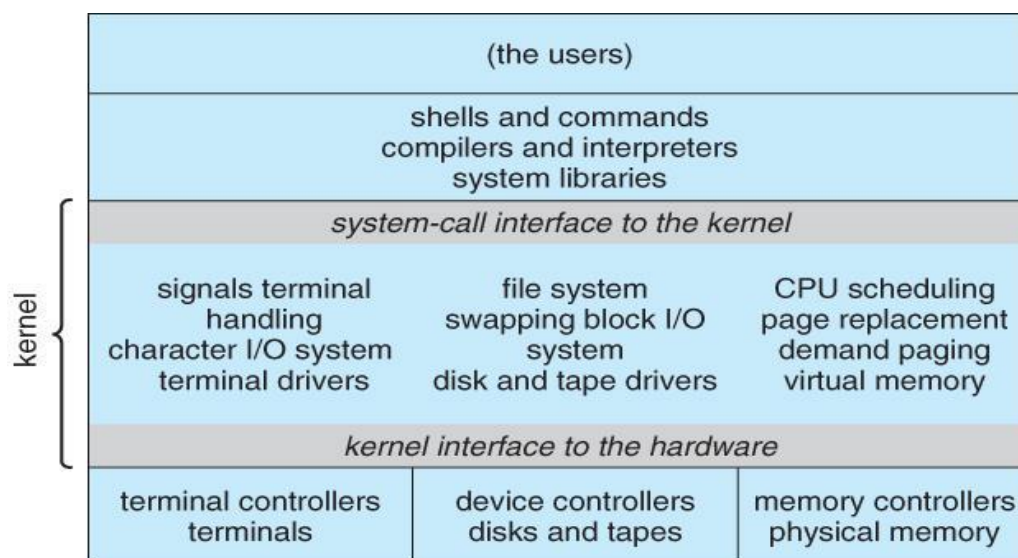
When DOS was originally written its developers had no idea how big and important it would eventually become. It was written by a few programmers in a relatively short amount of time, without the benefit of modern software engineering techniques, and then gradually grew over time to exceed its original expectations. It does not break the system into subsystems, and has no distinction

between user and kernel modes, allowing all programs direct access to the underlying hardware. ( Note that user versus kernel mode was not supported by the 8088 chip set anyway, so that really wasn't an option back then. )



**Figure - MS-DOS layer structure**

The original UNIX OS used a simple layered approach, but almost all the OS was in one big layer, not really breaking the OS down into layered subsystems:



**Figure - Traditional UNIX system structure**

### Layered Approach

- Another approach is to break the OS into a number of smaller layers, each of which rests on the layer below it, and relies solely on the services provided by the next lower layer.
- This approach allows each layer to be developed and debugged independently, with the assumption that all lower layers have already been debugged and are trusted to deliver proper services.
- The problem is deciding what order in which to place the layers, as no layer can call upon the services of any higher layer, and so many chicken-and-egg situations may arise.
- Layered approaches can also be less efficient, as a request for service from a higher layer has to filter through all lower layers before it reaches the HW, possibly with significant processing at each step.

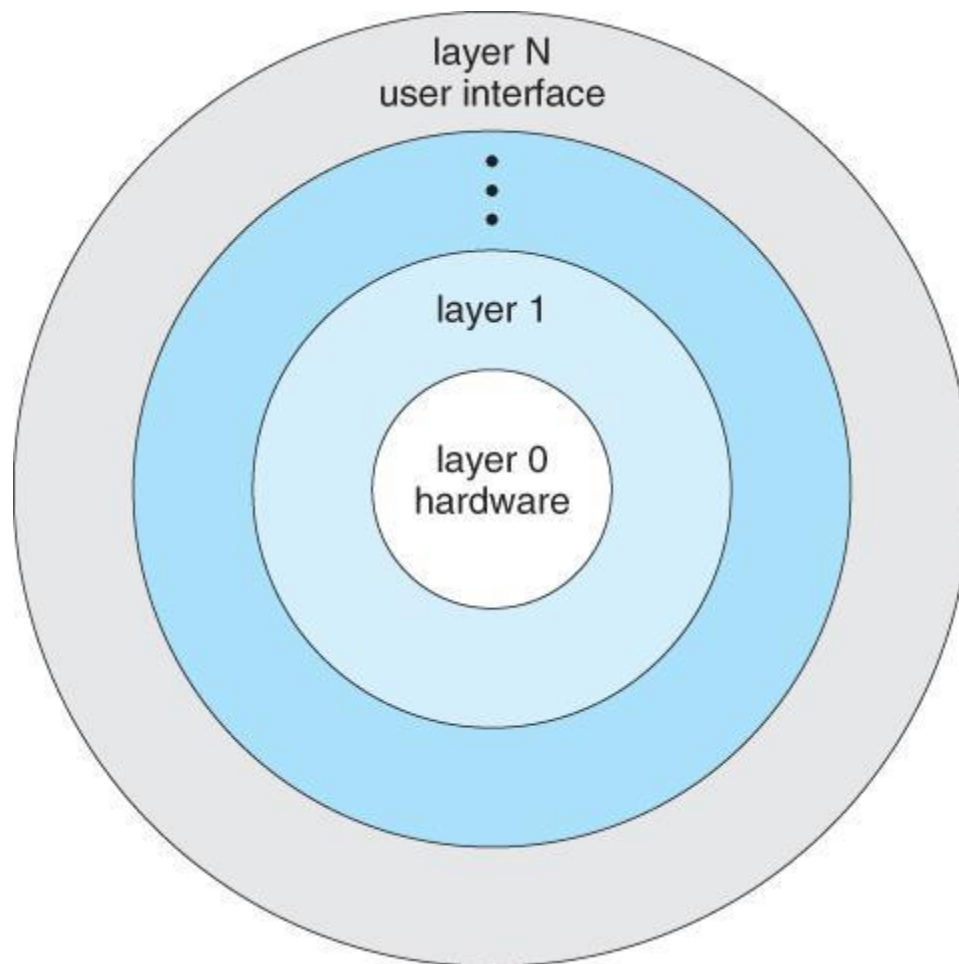


Figure - A layered operating system

## Microkernels

The basic idea behind micro kernels is to remove all non-essential services from the kernel, and implement them as system applications instead, thereby making the kernel as small and efficient as possible.

Most microkernels provide basic process and memory management, and message passing between other services, and not much more.

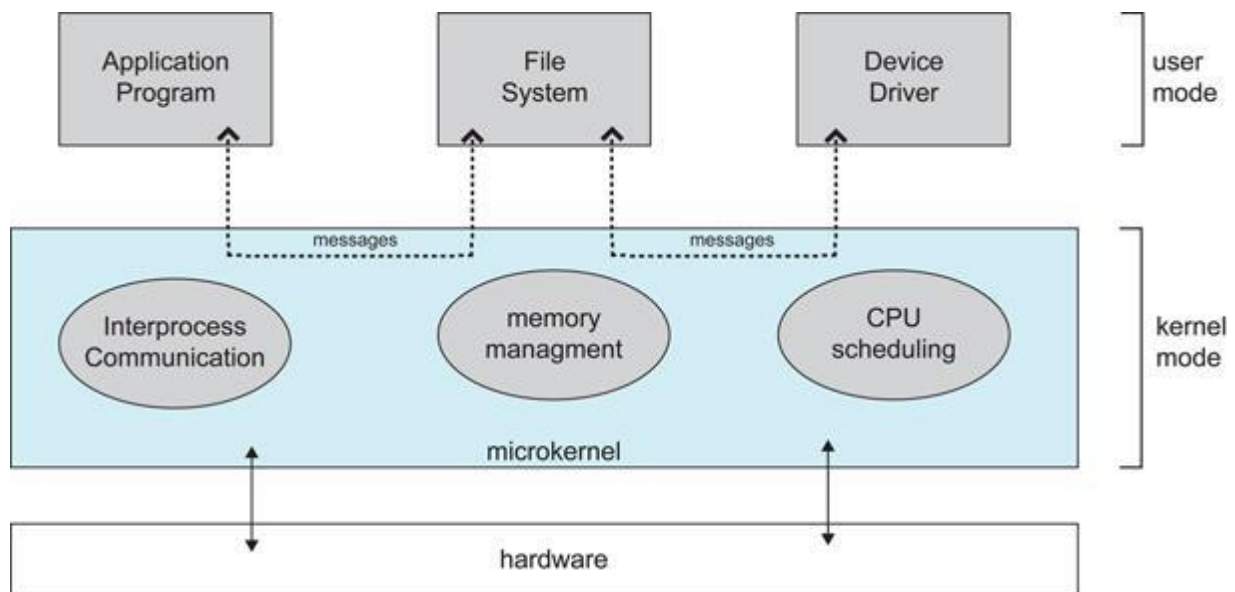
Security and protection can be enhanced, as most services are performed in user mode, not kernel mode.

System expansion can also be easier, because it only involves adding more system applications, not rebuilding a new kernel.

Mach was the first and most widely known microkernel, and now forms a major component of Mac OSX.

Windows NT was originally microkernel, but suffered from performance problems relative to Windows 95. NT 4.0 improved performance by moving more services into the kernel, and now XP is back to being more monolithic.

Another microkernel example is QNX, a real-time OS for embedded systems.



**Figure - Architecture of a typical microkernel**

## Modules

Modern OS development is object-oriented, with a relatively small core kernel and a set of **modules** which can be linked in dynamically. See for example the Solaris structure, as shown in Figure below. Modules are similar to layers in that each subsystem has clearly defined tasks and interfaces, but any module is free to contact any other module, eliminating the problems of going through multiple intermediary layers, as well as the chicken-and-egg problems.

The kernel is relatively small in this architecture, similar to microkernels, but the kernel does not have to implement message passing since modules are free to contact each other directly.

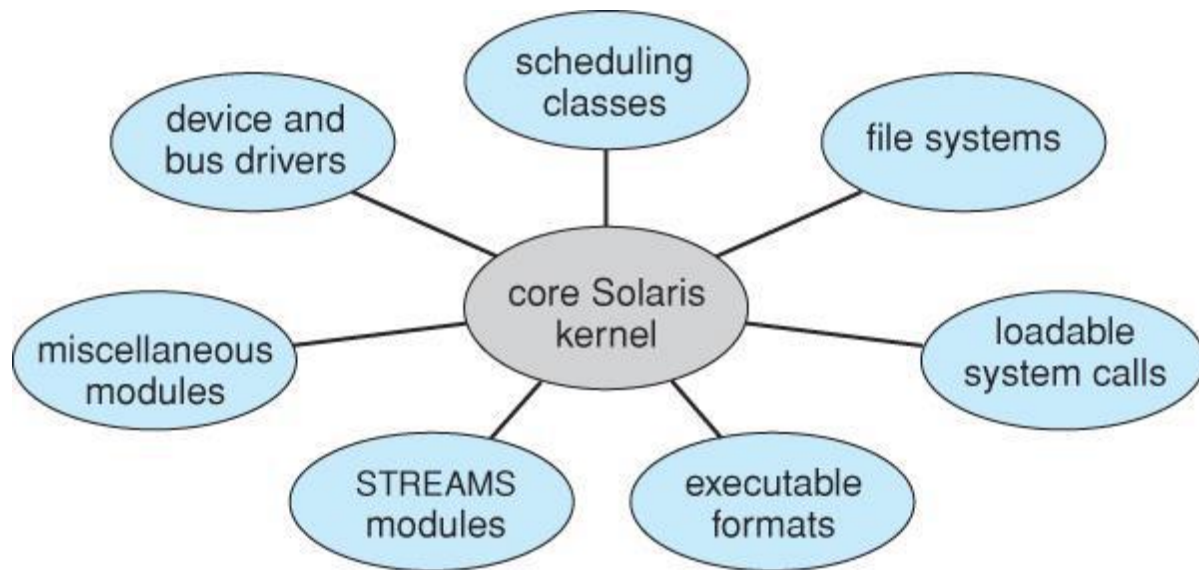


Figure - Solaris loadable modules

## Interrupts

- **Hardware Interrupts:** <https://www.youtube.com/watch?v=rnGVincwk30>
- **Software Interrupt:** <https://www.youtube.com/watch?v=KggMGZyiLnU>

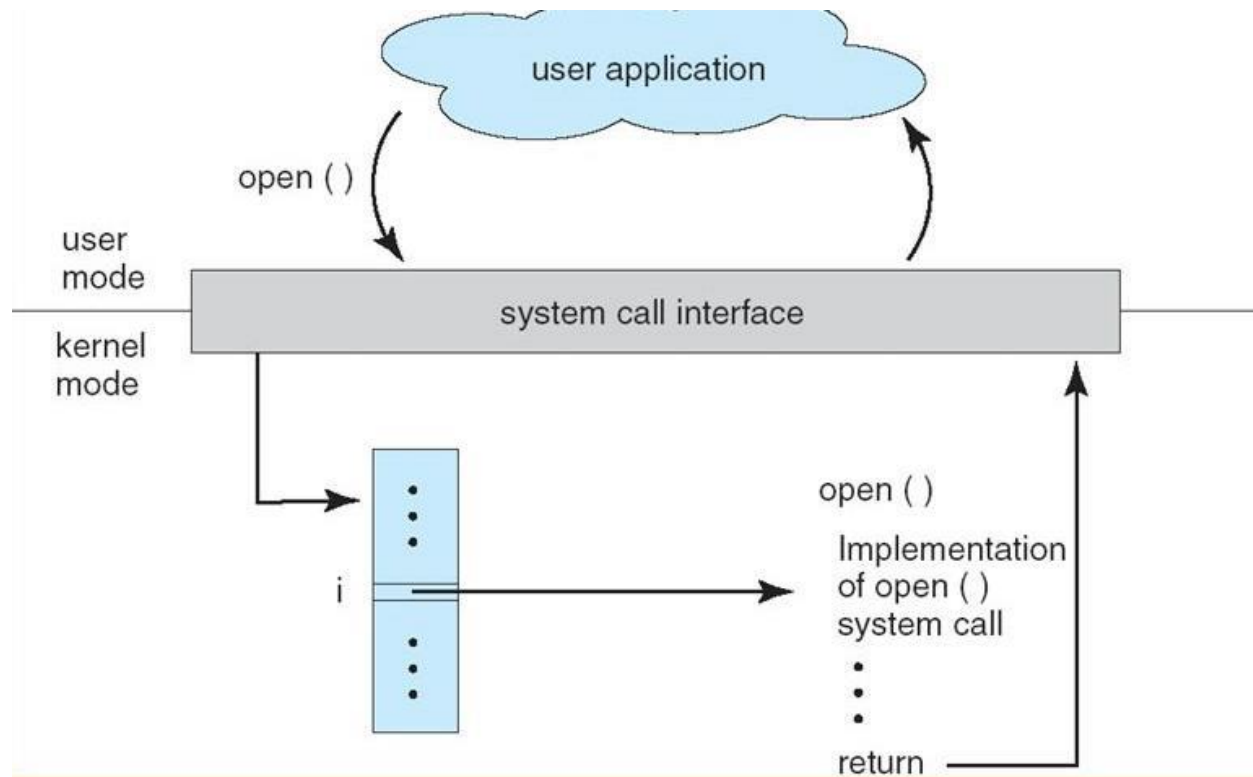
## System Call in OS

### What is System Call in Operating System?

A **system call** is a mechanism that provides the interface between a process and the operating system. It is a programmatic method in which a computer program requests a service from the kernel of the OS.

System call offers the services of the operating system to the user programs via API (Application Programming Interface). System calls are the only entry points for the kernel system.

### System Calls in Operating System

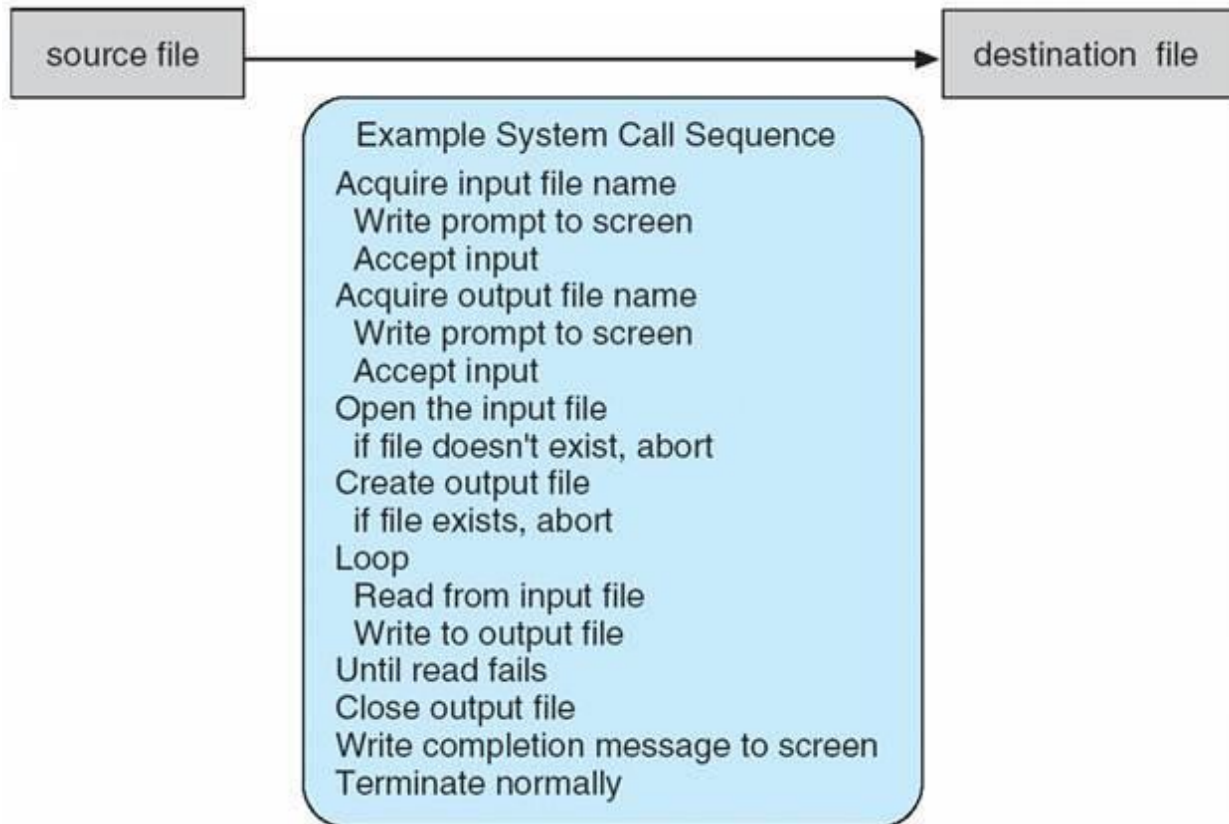


### Example of System Call

For example if we need to write a program code to read data from one file, copy that data into another file. The first information that the program requires is the name of the two files, the input and output files.

In an interactive system, this type of program execution requires some system calls by OS.

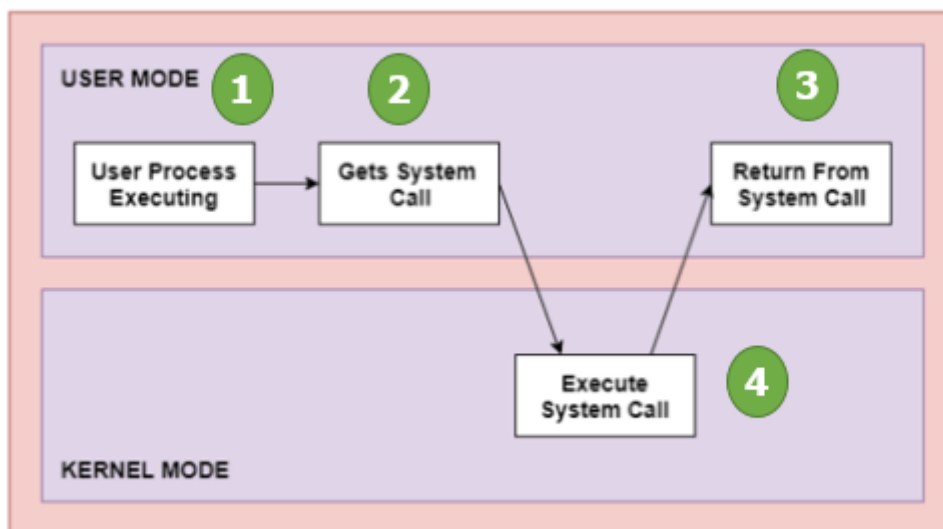




## How System Call Works?

Here are steps for System Call:

Architecture of the System Call



As you can see in the above-given diagram.

**Step 1)** The processes executed in the user mode till the time a system call interrupts it.

**Step 2)** After that, the system call is executed in the kernel-mode on a priority basis.

**Step 3)** Once system call execution is over, control returns to the user mode.,

**Step 4)** The execution of user processes resumed in Kernel mode.

## Why do you need System Calls in OS?

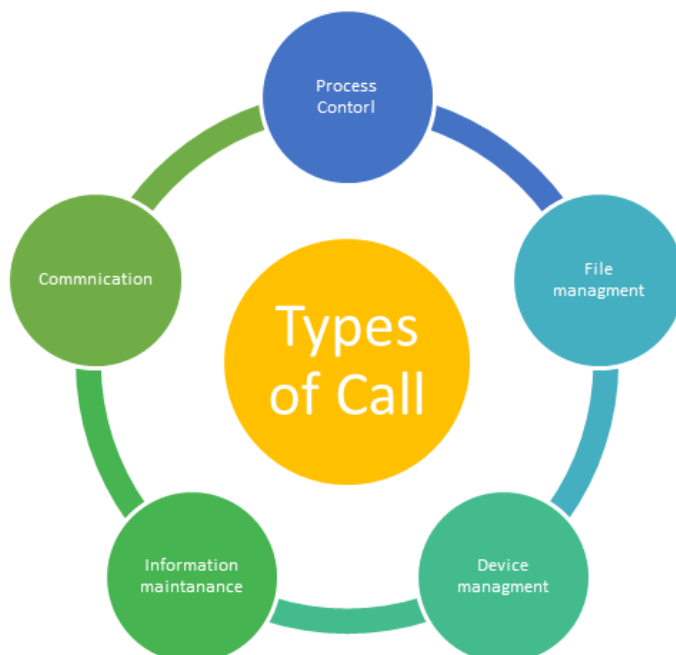
Following are situations which need system calls in OS:

- ❑ Reading and writing from files demand system calls.
- ❑ If a file system wants to create or delete files, system calls are required.
- ❑ System calls are used for the creation and management of new processes.
- ❑ Network connections need system calls for sending and receiving packets.
- ❑ Access to hardware devices like scanner, printer, need a system call.

## Types of System calls

Here are the five types of system calls used in OS:

- ❑ Process Control
- ❑ File Management
- ❑ Device Management
- ❑ Information Maintenance
- ❑ Communications



## **Process Control**

This system calls perform the task of process creation, process termination, etc.

Functions:

- End and Abort
- Load and Execute
- Create Process and Terminate Process
- Wait and Signed Event
- Allocate and free memory

## **File Management**

File management system calls handle file manipulation jobs like creating a file, reading, and writing, etc.

Functions:

- Create a file
- Delete file
- Open and close file
- Read, write, and reposition
- Get and set file attributes

## **Device Management**

Device management does the job of device manipulation like reading from device buffers, writing into device buffers, etc.

Functions

- Request and release device
- Logically attach/ detach devices
- Get and Set device attributes

## **Information Maintenance**

It handles information and its transfer between the OS and the user program.

Functions:

- Get or set time and date

- Get process and device attributes

### **Communication:**

These types of system calls are specially used for inter process communications.

Functions:

- Create, delete communications connections
- Send, receive message
- Help OS to transfer status information
- Attach or detach remote devices

### **Rules for passing Parameters for System Call**

Here are general common rules for passing parameters to the System Call:

- Parameters should be pushed on or popped off the stack by the operating system.
- Parameters can be passed in registers.
- When there are more parameters than registers, it should be stored in a block, and the block address should be passed as a parameter to a register.

### **Important System Calls Used in OS**

#### **wait()**

In some systems, a process needs to wait for another process to complete its execution. This type of situation occurs when a parent process creates a child process, and the execution of the parent process remains suspended until its child process executes.

The suspension of the parent process automatically occurs with a wait() system call. When the child process ends execution, the control moves back to the parent process.

#### **fork()**

Processes use this system call to create processes that are a copy of themselves. With the help of this system Call parent process creates a child process, and the execution of the parent process will be suspended till the child process executes.

## **exec()**

This system call runs when an executable file in the context of an already running process that replaces the older executable file. However, the original process identifier remains as a new process is not built, but stack, data, heap, data, etc. are replaced by the new process.

## **kill():**

The kill() system call is used by OS to send a termination signal to a process that urges the process to exit. However, a kill system call does not necessarily mean killing the process and can have various meanings.

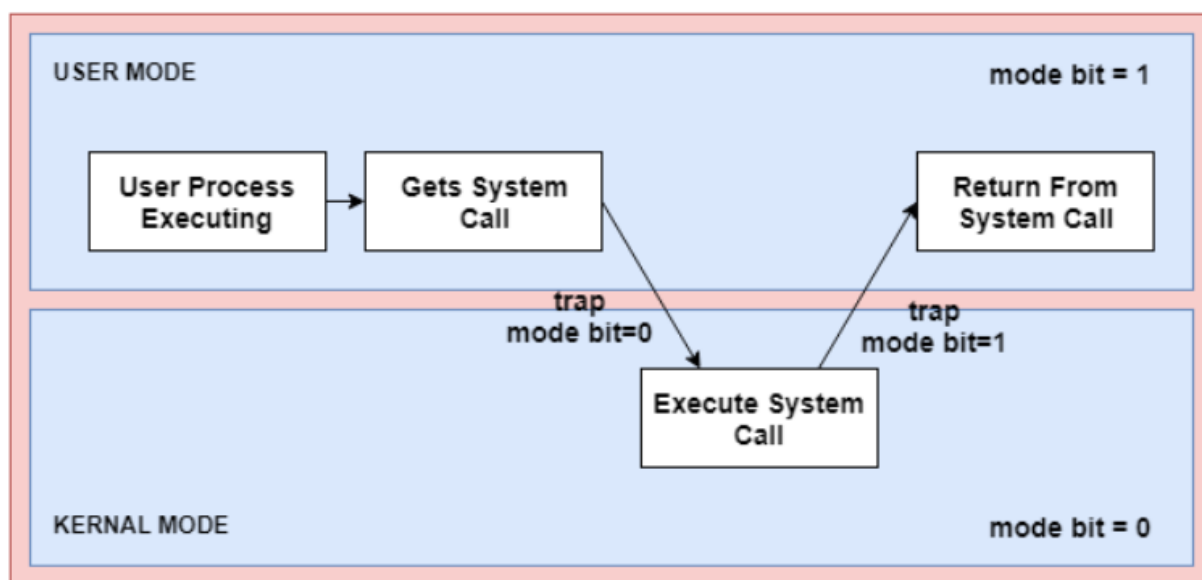
## **exit():**

The exit() system call is used to terminate program execution. Specially in the multi-threaded environment, this call defines that the thread execution is complete. The OS reclaims resources that were used by the process after the use of exit() system call.

## **Dual-Mode / Multi-Mode Operation in Operating System**

There are two modes of operation in the operating system to make sure it works correctly. These are user mode and kernel mode.

A diagram that illustrates the transition from user mode to kernel mode and back again is as follows –



The following are the modes –

- **User Mode:**

The system is in user mode when the operating system is running a user application such as handling a text editor. The transition from user mode to kernel mode occurs when the application requests the help of operating system or an interrupt or a system call occurs.

The mode bit is set to 1 in the user mode. It is changed from 1 to 0 when switching from user mode to kernel mode.

- **Kernel Mode**

The system starts in kernel mode when it boots and after the operating system is loaded, it executes applications in user mode. There are some privileged instructions that can only be executed in kernel mode. These are interrupt instructions, input output management etc. If the privileged instructions are executed in user mode, it is illegal and a trap is generated.

The mode bit is set to 0 in the kernel mode. It is changed from 0 to 1 when switching from kernel mode to user mode.

The concept of modes of operation in operating system can be extended beyond the dual mode. This is known as the multimode system. In those cases the more than 1 bit is used by the CPU to set and handle the mode.

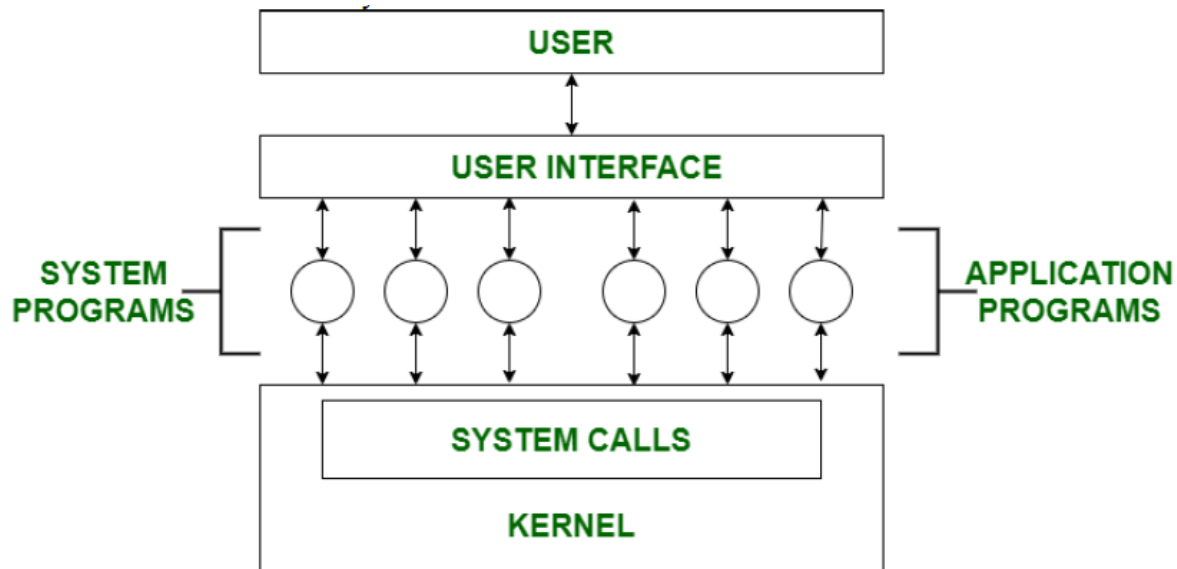
## System Programs in Operating System

**System Programming** can be defined as act of building Systems Software using System Programming Languages. According to Computer Hierarchy, one which comes at last is Hardware. Then it is Operating System, System Programs, and finally Application Programs. Program Development and Execution can be done conveniently in System Programs. Some of System Programs are simply user interfaces, others are complex. It traditionally lies between user interface and system calls. System Programs can be divided into these categories :

**1. File Management** – A file is a collection of specific information stored in memory of computer system. File management is defined as process of manipulating files in computer system, it management includes process of creating, modifying and deleting files.

- It helps to create new files in computer system and placing them at specific locations.
- It helps in easily and quickly locating these files in computer system.
- It makes process of sharing of files among different users very easy and user friendly.
- It helps to stores files in separate folders known as directories.
- These directories help users to search file quickly or to manage files according to their types or uses.
- It helps user to modify data of files or to modify he name of file in directories.

**2. Status Information** – Information like date, time amount of available memory, or disk space is asked by some of users. Other's providing detailed performance, logging and debugging information which is more complex. All this information is formatted and displayed on output devices or printed. Terminal or other output devices or files or a window of GUI is used for showing output of programs.



**3. File Modification** – For modifying contents of files we use this. For Files stored on disks or other storage devices we used different types of editors. For searching contents of files or perform transformations of files we use special commands.

**4. Programming-Language support** – For common programming languages we use *Compilers, Assemblers, Debuggers and interpreters* which are already provided to user. It provides all support to users. We can run any programming languages. All languages of importance are already provided.

**5. Program Loading and Execution** – When program is ready after Assembling and compilation, it must be loaded into memory for execution. A *loader* is part of an operating system that is responsible for loading programs and libraries. It is one of essential stages for starting a program. Loaders, relocatable loaders, linkage editors and Overlay loaders are provided by system.

**6. Communications** – Virtual connections among processes, users and computer systems are provided by programs. User can send messages to other user on their screen, User can send e-mail, browsing on web pages, remote login, transformation of files from one user to another.

# Computer System Structure

Computer system can be divided into four components:

**Hardware** – provides basic computing resources

CPU, memory, I/O devices

**Operating system:** Controls and coordinates use of hardware among various applications and users

**Application programs** – define the ways in which the system resources are used to solve the computing problems of the users: Word processors, compilers, web browsers, database systems, video games

**Users:** People, machines, other computers

